

Multimodal LSAT Ingestion Pipeline

Executive architecture summary

This design connects your existing `lsat-pipeline/` folders into two ingestion lanes that converge on one canonical LSAT schema. The image lane accepts iPhone photos or screenshots, performs image quality checks, runs deterministic OCR, stores raw OCR artifacts in `data/ocr_output/`, then normalizes the text. The text lane accepts generated or digital text from `data/raw_text/` and sends it directly into the same normalization and mapping layer. Both lanes merge into `data/normalized/`, then run through classification, validation, persistence, and review routing.

The guiding principle is that OCR and generated text should differ only in the front half of the pipeline. After `src/normalize/canonical_mapper.py`, every downstream module should receive the same record shape, with `source.modality` preserving whether the record came from image OCR or digital text.

Based on the provided tree, the best first implementation is not a full hierarchical transformer. The smallest robust version should implement deterministic OCR, canonical mapping, validation gates, and a chunk-capable classifier interface. Straight BERT classification is acceptable only for short LSAT units where the task signal fits inside the first window. Chunk-based classification is the best MVP tradeoff when passages plus question context exceed vanilla BERT limits. Hierarchical classification is worth adding after you measure token lengths and confirm that labels depend on evidence distributed across the whole passage.

Assumptions and safer alternatives

- **Assumption:** existing code exists in the listed Python files: Because the actual repo was not accessible, reuse recommendations are based on your folder and file names. Safer alternative: run this audit again against the repo once GitHub or the local project shell is available.
- **Assumption:** prior iPhone OCR/Live Text workflow remains useful: Past context indicates you previously favored iPhone OCR/Live Text as a local/offline front end for LSAT capture. Safer alternative: keep both Apple Live Text imports and Tesseract/PyTesseract imports behind `ocr_router.py`.
- **Assumption:** persistence may start as JSONL or SQLite before PostgreSQL: Your folder tree does not show a DB layer yet. Safer alternative: implement `src/persist/` with a filesystem adapter first, then add PostgreSQL when API workflows need querying.
- **Assumption:** LSAT labels are initially section/type labels rather than fine-grained reasoning diagnostics: If your classification goals include flaw type, question family, RC passage topic, and difficulty, use multilabel support from the start.

Updated project tree

```
lsat-pipeline/
├── docs/
│   ├── architecture.md
│   ├── api_contract.md
│   ├── data_dictionary.md
│   ├── prompts/
│   │   ├── metadata_extraction.md
│   │   └── split_repair.md
│   └── schemas/
│       ├── canonical_lsat_record.schema.json
│       ├── ingestion_manifest.schema.json
│       └── validation_report.schema.json
├── data/
│   └── raw_images/
```

```
| | | .gitkeep
| | raw_text/
| | | .gitkeep
| | ocr_output/
| | | text/
| | | tsv/
| | | manifests/
| | normalized/
| | | records/
| | | manifests/
| | review_queue/
| | | low_ocr_confidence/
| | | malformed_split/
| | | missing_fields/
| | | duplicate_candidates/
| | | ambiguous_classification/
| models/
| | bert/
| | tokenizers/
| | label_maps/
| src/
| | ingest/
| | | __init__.py
| | | image_loader.py
| | | text_loader.py
| | | manifest.py
| | ocr/
| | | __init__.py
| | | ocr_router.py
| | | image_quality.py
| | | extract_text.py
| | normalize/
| | | __init__.py
| | | clean_text.py
| | | split_lsat.py
| | | canonical_mapper.py
| | classify/
| | | __init__.py
| | | classify_document.py
| | | chunking.py
| | | hierarchical.py
| | | label_schema.py
| | validate/
| | | __init__.py
| | | confidence_checks.py
| | | dedupe.py
| | | review_routing.py
| | persist/
| | | __init__.py
| | | filesystem_store.py
| | | postgres_store.py
| | | models.py
| | review/
| | | __init__.py
| | | queue.py
| | api/
| | | __init__.py
| | | main.py
| | | routes.py
| tests/
| | fixtures/
| | | sample_image_manifest.json
| | | sample_ocr_text.txt
| | | sample_generated_text.txt
| | test_image_lane.py
| | test_text_lane.py
```

```

|   ├── test_canonical_mapper.py
|   ├── test_validation.py
|   └── test_classification_chunking.py
└── docker/
    ├── Dockerfile
    ├── docker-compose.yml
    └── entrypoint.sh

```

Proposed folder cleanup

- **Keep** `data/raw_images/`: Store unmodified iPhone photos/screenshots plus sidecar manifests. Never overwrite originals.
- **Keep** `data/raw_text/`: Store generated text, copied Apple Live Text output, exported Notes text, and synthetic LSAT-style content.
- **Keep** `data/ocr_output/`: Split into `text/`, `tsv/`, and `manifests/` so plain OCR text, word-level confidence data, and run metadata do not mix.
- **Keep** `data/normalized/`: Treat as the canonical merge point. Every file here should validate against `canonical_lsat_record.schema.json`.
- **Keep** `data/review_queue/`: Use reason-specific subfolders so bad OCR, malformed splits, missing fields, duplicates, and ambiguous labels can be reviewed independently.
- **Add** `src/persist/`: Persistence is a separate module, not an API responsibility.
- **Add** `src/review/`: Review queue operations should not live inside validation alone. Validation decides what is wrong; review persists the item for human correction.
- **Add** `docs/api_contract.md` **and** `docs/data_dictionary.md`: These prevent the API, schema, and downstream retrieval layer from drifting apart.

End-to-end architecture diagram

```

flowchart TD
    A[data/raw_images/] --> B[src/ingest/image_loader.py]
    B --> C[src/ocr/image_quality.py]
    C --> D[src/ocr/ocr_router.py]
    D --> E[src/ocr/extract_text.py]
    E --> F[data/ocr_output/]
    F --> G[src/normalize/clean_text.py]

    H[data/raw_text/] --> I[src/ingest/text_loader.py]
    I --> G

    G --> J[src/normalize/split_lsat.py]
    J --> K[src/normalize/canonical_mapper.py]
    K --> L[data/normalized/]

    L --> M[src/classify/chunking.py]
    M --> N[src/classify/classify_document.py]
    N --> O[src/validate/confidence_checks.py]
    O --> P[src/validate/dedupe.py]
    P --> Q[src/validate/review_routing.py]
    Q --> R[data/review_queue/]
    R --> S[src/persist/filesystem_store.py or postgres_store.py]
    S --> T[src/api/routes.py]

```

Module dependency map

Image lane

File	Imports or calls	Depends on	Output
<code>src/ingest/image_loader.py</code>	<code>src.ingest.manifest</code> , <code>pathlib</code> , <code>image</code> metadata utilities	<code>data/raw_images/</code>	IngestionManifest records for images
<code>src/ocr/image_quality.py</code>	OpenCV or PIL utilities	Image paths from <code>image_loader.py</code>	Quality report: blur, skew, resolution, orientation, crop warnings
<code>src/ocr/ocr_router.py</code>	<code>image_quality.check_image_quality</code> , <code>extract_text.run_tesseract</code> , optional Apple Live Text import adapter	Image manifest and config	OCR job decision and OCR result path
<code>src/ocr/extract_text.py</code>	PyTesseract/Tesseract, PIL/OpenCV	Image path, OCR config	.txt, .tsv, average confidence, word confidence rows
<code>src/normalize/clean_text.py</code>	regex utilities, OCR correction rules	OCR .txt or generated text	normalized plain text

Tesseract/PyTesseract should use structured OCR output rather than only plain text. PyTesseract exposes word-level text and confidence through `image_to_data`, which supports downstream confidence aggregation and review routing ([Stack Overflow](#)).

Text lane

File	Imports or calls	Depends on	Output
<code>src/ingest/text_loader.py</code>	<code>src.ingest.manifest</code> , text file parsing	<code>data/raw_text/</code>	text manifests and raw text payloads
<code>src/normalize/clean_text.py</code>	same cleaning functions used by image lane	text payload	normalized plain text
<code>src/normalize/split_lsats.py</code>	LSAT regex grammar and repair hooks	normalized plain text	passage, stimulus, stem, answer choices, explanation segments
<code>src/normalize/canonical_mapper.py</code>	schema models	split segments plus source metadata	canonical LSAT JSON

Shared downstream lane

File	Imports or calls	Depends on	Output
<code>src/normalize/split_lsats.py</code>	<code>clean_text.normalize_text</code> output	cleaned text	structured LSAT segments
<code>src/normalize/canonical_mapper.py</code>	<code>src.classify.label_schema</code> , schema constants	segments, manifest	canonical record
<code>src/classify/chunking.py</code>	tokenizer from <code>models/tokenizers/</code>	canonical record text fields	chunks with token spans
<code>src/classify/classify_document.py</code>	<code>chunking</code> , <code>label_schema</code> , model from <code>models/bert/</code>	canonical record	label predictions and confidence
<code>src/classify/hierarchical.py</code>	chunk embeddings, aggregation model	chunk representations	document-level decision

File	Imports or calls	Depends on	Output
src/validate/confidence_checks.py	OCR metrics, classifier scores, schema fields	canonical record plus predictions	validation report
src/validate/dedupe.py	hash, fuzzy matching, embeddings optional	canonical record	duplicate candidate report
src/validate/review_routing.py	validation report	canonical record and validation report	review queue destination
src/review/queue.py	filesystem or DB adapter	review destination	persisted review item
src/persist/filesystem_store.py	JSONL/Parquet writer	canonical record	local persistence
src/persist/postgres_store.py	SQLAlchemy or psycopg	canonical record	database rows
src/api/routes.py	ingest, normalize, classify, validate, persist services	FastAPI	HTTP endpoints

Canonical LSAT schema

Use this schema for both OCR-derived records and generated-text records. Source details preserve provenance without changing downstream retrieval or classification code.

```
{
  "record_id": "uuid",
  "schema_version": "1.0.0",
  "source": {
    "modality": "image|digital_text|generated_text",
    "source_file": "string",
    "source_uri": "string|null",
    "page_ref": "string|null",
    "image_ref": "string|null",
    "capture_device": "iPhone|unknown|null",
    "ocr_engine": "tesseract|apple_live_text|none|null",
    "ocr_run_id": "string|null",
    "created_at": "datetime"
  },
  "lsat": {
    "exam": "LSAT",
    "prep_test": "string|null",
    "section": "logical_reasoning|reading_comprehension|logic_games|unknown",
    "section_number": "integer|null",
    "question_number": "integer|null",
    "passage_id": "string|null",
    "question_type": "string|null",
    "difficulty": "string|null"
  },
  "content": {
    "passage": "string|null",
    "stimulus": "string|null",
    "question_stem": "string|null",
    "answer_choices": [
      {"label": "A", "text": "string"},
      {"label": "B", "text": "string"},
      {"label": "C", "text": "string"},
      {"label": "D", "text": "string"},
      {"label": "E", "text": "string"}
    ],
    "correct_answer": "A|B|C|D|E|null",
    "explanation": "string|null",
  }
}
```

```

    "raw_text": "string",
    "normalized_text": "string"
  },
  "ocr": {
    "mean_confidence": "number|null",
    "min_line_confidence": "number|null",
    "low_confidence_spans": [
      {"text": "string", "confidence": "number", "bbox": [0, 0, 0, 0]}
    ],
    "image_quality": {
      "blur_score": "number|null",
      "skew_degrees": "number|null",
      "resolution_dpi": "number|null",
      "quality_flags": ["string"]
    }
  },
  "classification": {
    "strategy": "straight|chunked|hierarchical|long_context",
    "labels": [
      {"name": "string", "score": "number", "label_type": "section|question_type|topic|skill|diffic
    ],
    "ambiguous": "boolean",
    "token_count": "integer",
    "chunks": [
      {"chunk_id": "string", "start_token": "integer", "end_token": "integer", "score": "number|nu
    ]
  },
  "validation": {
    "status": "valid|needs_review|rejected",
    "errors": ["string"],
    "warnings": ["string"],
    "duplicate_candidates": ["record_id"],
    "review_reason": "string|null"
  }
}

```

API contract

POST /ingest/image

Accepts an image file or image path manifest. It writes the raw file under `data/raw_images/`, runs image quality checks, runs OCR, writes OCR artifacts under `data/ocr_output/`, maps to the canonical schema, validates, classifies, and persists.

```

{
  "source_file": "data/raw_images/PT93_RC_p1.jpg",
  "metadata": {
    "prep_test": "PT93",
    "section": "reading_comprehension",
    "page_ref": "p1",
    "capture_device": "iPhone"
  },
  "run_ocr": true,
  "classify": true
}

```

POST /ingest/text

Accepts generated text or digital text. It skips OCR and uses the same cleaner, splitter, mapper, classifier, validator, review router, and persistence path.

```
{
  "source_file": "data/raw_text/generated_lr_001.txt",
  "modality": "generated_text",
  "metadata": {
    "section": "logical_reasoning",
    "source": "synthetic_generation"
  },
  "classify": true
}
```

GET /records/{record_id}

Returns one canonical record from the persistence layer.

GET /review-queue

Returns review items grouped by reason: `low_ocr_confidence`, `malformed_split`, `missing_fields`, `duplicate_candidates`, and `ambiguous_classification`.

POST /review-queue/{record_id}/resolve

Accepts corrected fields, writes an updated canonical record, and marks the review item resolved.

Database schema

Start with SQLite or PostgreSQL tables that mirror the canonical schema but avoid over-normalizing too early.

```
CREATE TABLE lsat_records (
  record_id UUID PRIMARY KEY,
  schema_version TEXT NOT NULL,
  modality TEXT NOT NULL,
  source_file TEXT NOT NULL,
  source_uri TEXT,
  page_ref TEXT,
  image_ref TEXT,
  prep_test TEXT,
  section TEXT NOT NULL,
  section_number INTEGER,
  question_number INTEGER,
  passage_id TEXT,
  question_type TEXT,
  difficulty TEXT,
  passage TEXT,
  stimulus TEXT,
  question_stem TEXT,
  correct_answer TEXT,
  explanation TEXT,
  raw_text TEXT NOT NULL,
  normalized_text TEXT NOT NULL,
  mean_ocr_confidence REAL,
  token_count INTEGER,
  classification_strategy TEXT,
  validation_status TEXT NOT NULL,
  review_reason TEXT,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL
)
```

```

);

CREATE TABLE answer_choices (
    id UUID PRIMARY KEY,
    record_id UUID NOT NULL REFERENCES lsat_records(record_id),
    label TEXT NOT NULL,
    text TEXT NOT NULL,
    UNIQUE(record_id, label)
);

CREATE TABLE classification_labels (
    id UUID PRIMARY KEY,
    record_id UUID NOT NULL REFERENCES lsat_records(record_id),
    name TEXT NOT NULL,
    score REAL NOT NULL,
    label_type TEXT NOT NULL
);

CREATE TABLE validation_events (
    id UUID PRIMARY KEY,
    record_id UUID NOT NULL REFERENCES lsat_records(record_id),
    severity TEXT NOT NULL,
    code TEXT NOT NULL,
    message TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL
);

CREATE TABLE duplicate_candidates (
    id UUID PRIMARY KEY,
    record_id UUID NOT NULL REFERENCES lsat_records(record_id),
    candidate_record_id UUID NOT NULL REFERENCES lsat_records(record_id),
    method TEXT NOT NULL,
    score REAL NOT NULL
);

CREATE TABLE review_items (
    id UUID PRIMARY KEY,
    record_id UUID NOT NULL REFERENCES lsat_records(record_id),
    reason TEXT NOT NULL,
    status TEXT NOT NULL,
    payload JSONB,
    created_at TIMESTAMP NOT NULL,
    resolved_at TIMESTAMP
);

```

Pipeline pseudocode

```

def run_image_lane(image_path: str, metadata: dict) -> CanonicalRecord:
    manifest = image_loader.load_image(image_path, metadata)
    quality = image_quality.check_image_quality(manifest)
    ocr_result = ocr_router.run(manifest, quality)
    cleaned = clean_text.normalize(ocr_result.text)
    segments = split_lsat.split(cleaned, hints=metadata)
    record = canonical_mapper.map_to_record(
        source_manifest=manifest,
        raw_text=ocr_result.text,
        normalized_text=cleaned,
        segments=segments,
        ocr=ocr_result,
        image_quality=quality,
    )
    return run_shared_downstream(record)

```



```

def run_text_lane(text_path: str, metadata: dict) -> CanonicalRecord:
    manifest, raw_text = text_loader.load_text(text_path, metadata)
    cleaned = clean_text.normalize(raw_text)
    segments = split_lsat.split(cleaned, hints=metadata)
    record = canonical_mapper.map_to_record(
        source_manifest=manifest,
        raw_text=raw_text,
        normalized_text=cleaned,
        segments=segments,
        ocr=None,
        image_quality=None,
    )
    return run_shared_downstream(record)

def run_shared_downstream(record: CanonicalRecord) -> CanonicalRecord:
    chunks = chunking.make_chunks(record)
    prediction = classify_document.predict(record, chunks)
    record.classification = prediction

    validation_report = confidence_checks.validate(record)
    duplicate_report = dedupe.find_candidates(record)
    record.validation = review_routing.decide(validation_report, duplicate_report)

    if record.validation.status == "needs_review":
        queue.add(record, reason=record.validation.review_reason)

    persist.save(record)
    return record

```

Validation and quality checks

- **OCR confidence:** route to review when mean confidence is below threshold, when any required line has very low confidence, or when low-confidence spans overlap answer choices or question stems. PyTesseract can expose confidence data through `image_to_data`, so keep `.tsv` or DataFrame-equivalent output alongside plain OCR text ([Stack Overflow](#)).
- **Image quality:** flag blur, skew, low resolution, bad crop, missing page area, and orientation mismatch before OCR. Do not discard automatically unless the file is unreadable.
- **Missing fields:** require `section`, at least one of `stimulus` or `passage`, `question_stem` for question records, and answer choices A through E unless the item is passage-only.
- **Malformed question splits:** flag records with fewer than five choices, repeated labels, answer choices embedded in the stem, multiple question numbers in one record, or an explanation attached to the wrong question.
- **Duplicate detection:** compute exact hashes on normalized text, fuzzy hashes on cleaned text, and optional embedding similarity for near-duplicates across OCR and generated text.
- **Ambiguous classification:** route to review when top label score is below threshold, top two labels are too close, multilabel outputs conflict, or section predicted by model conflicts with metadata.
- **Validation loop:** treat LLM-based extraction as a repair and review aid, not the first source of truth. Deterministic parsing plus explicit validation should run first, then LLM repair should operate only on flagged records.

Model recommendations by stage

Stage	Recommended default	Deterministic or LLM	Reason
Image quality	OpenCV/PIL checks	Deterministic	Blur, skew, orientation, and resolution should be reproducible.

Stage	Recommended default	Deterministic or LLM	Reason
OCR	Apple Live Text import or Tesseract 5/PyTesseract	Deterministic	OCR needs stable output, confidence scores, and rerunnable artifacts. Tesseract confidence should be treated as a review signal, not perfect truth.
Text cleaning	Regex plus LSAT-specific normalization rules	Deterministic	Fix spacing, labels, OCR confusions, and line breaks reproducibly.
Segmentation	Regex/state machine first	Deterministic with LLM repair fallback	LSAT answer labels and stems are structured enough for rule-based parsing. Use LLM only when validation flags malformed splits.
Metadata extraction	filename/manifest first, LLM fallback	Hybrid	PrepTest, section, page, and image reference should come from filenames or metadata when possible.
Classification	BERT-compatible classifier with chunking interface	ML model	Start with straight or chunked classification depending on measured token lengths.
Validation	Schema checks, confidence thresholds, duplicate checks	Deterministic	Validation should be testable and auditable.
Review queue	Rule-based routing	Deterministic	Human review needs clear reasons, not opaque prompt output.

Classification strategy recommendation

BERT-style models commonly use a 512-token maximum sequence length, which is why long-document workflows often truncate, chunk, or switch architectures instead of feeding the full document directly ([Stack Overflow](#)). Longformer is designed for long-document transformer use cases and uses local attention patterns rather than standard full attention for every token pair ([Hugging Face Longformer documentation](#)). BigBird extends transformer models such as BERT to longer sequences using sparse attention, but its own repository notes that for sequence lengths below 1024, full attention can be preferable because sparse attention may not help ([Google Research BigBird](#)).

Recommendation matrix

Strategy	Use when	LSAT fit	Memory cost	Complexity	Robustness	Recommendation
Straight truncated BERT	The unit is short, the answer signal appears early, and token counts fit within one input window	Good for short LR stimulus plus question stem, especially section or broad question-type labels	Lowest	Lowest	Weak if key evidence is late or split across passage and choices	Use for MVP only after measuring token lengths
Head-tail BERT	The beginning and ending carry most signal, such as passage intro plus answer/explanation tail	Useful when explanations are long but the important question/stimulus context is near the front	Low	Low	Better than pure truncation but can miss middle evidence	Add as a simple baseline
Chunk-based classification	Many examples exceed one BERT window, but labels can be inferred from local chunks	Best near-term tradeoff for RC passages plus question context	Medium	Medium	Good if max/mean pooling over chunk logits is enough	Recommended default after MVP
Hierarchical classification	Signal is distributed across passage, question, choices, and explanation	Best for RC passage-level topic, reasoning skill, and nuanced multilabel classification	Higher	Higher	Strongest BERT-family approach for distributed evidence	Worth it after token audit and baseline metrics

Strategy	Use when	LSAT fit	Memory cost	Complexity	Robustness	Recommendation
Longformer or BigBird	Full context regularly exceeds BERT limits and you have enough labeled data or can fine-tune reliably	Good for full RC passage sets or records with explanations	Medium to high	Medium	Strong for long context, but heavier than chunked BERT	Consider after measuring token lengths and label needs

Practical recommendation

For your first working version, implement `chunking.py` even if `classify_document.py` initially runs straight BERT. The reason is that the interface should not change when you move from short LR items to RC passages. The classifier should accept `CanonicalRecord`, produce `token_count`, and select one of these strategies:

```
if token_count <= max_model_tokens and classification_target in ["section", "short_question_type"]:
    strategy = "straight"
elif token_count <= head_tail_safe_limit and signal_expected_at_edges:
    strategy = "head_tail"
elif token_count <= long_context_model_limit and long_context_model_enabled:
    strategy = "long_context"
else:
    strategy = "chunked"
```

Do not extend vanilla BERT positional embeddings as the main plan. Use chunking first, then evaluate Longformer or BigBird if full-context labels outperform chunk pooling on your measured corpus.

Token-length measurement plan

Before choosing the final classifier, add a script or test that tokenizes every normalized record with the target tokenizer and reports:

- percentage of records under 128, 256, 384, 512, 768, 1024, 2048, and 4096 tokens;
- separate histograms for LR, RC passage-only, RC question-with-passage, generated text, and OCR text;
- percentage of records where answer choices or explanations fall outside the first 512 tokens;
- average token count by classification goal.

The decision rule should be:

- if 80 to 90 percent of target units fit within the selected BERT window and labels do not require late text, straight BERT is acceptable for MVP;
- if many records exceed the window but local chunks carry the label, use chunk-based classification;
- if performance drops because evidence is spread across passage, stem, choices, and explanation, move to hierarchical aggregation;
- if most records exceed 1024 to 4096 tokens and full-context reasoning matters, benchmark Longformer or BigBird.

Risks and failure modes

- **OCR overconfidence:** OCR confidence can be useful but is not absolute. Route low-confidence spans to review, but also spot-check high-confidence OCR.
- **Choice-label corruption:** OCR often confuses (A) with stray punctuation or line starts. Use answer-label-specific validation.
- **Multiple questions in one image:** Screenshots may contain two or more questions. `split_lsats.py` should support one-to-many outputs.

- **Passage/question linkage errors:** RC passages and questions can be separated across screenshots. Use `passage_id`, `page_ref`, and `source filename` conventions.
- **Generated text drift:** Generated LSAT-style content may not match real LSAT formatting. Keep modality in the schema and avoid mixing generated examples into evaluation without labels.
- **Duplicate pollution:** The same problem can enter through OCR and digital text. Use normalized text hashes and fuzzy matching.
- **Classifier leakage:** If explanations include correct answers, classifier tasks can accidentally learn answer/explanation artifacts. Define classification inputs per task.
- **Overusing LLM extraction:** A single prompt will fail on edge cases. Use deterministic parsing, validation, targeted repair prompts, and review queues.
- **Docker OCR dependencies:** Tesseract language data, system packages, OpenCV, and PIL must be pinned in `docker/Dockerfile`.

First implementation sprint: smallest working version

Goal

Build one runnable path for each lane:

```
raw image -> OCR -> normalize -> canonical JSON -> validate -> review or persist
raw text -> normalize -> canonical JSON -> validate -> review or persist
```

Sprint tasks

1. Schema first

- Create `docs/schemas/canonical_lsats_record.schema.json`.
- Add Python dataclasses or Pydantic models in `src/persist/models.py`.
- Add fixture records in `tests/fixtures/`.

2. Ingestion manifest

- Add `src/ingest/manifest.py`.
- Make `image_loader.py` and `text_loader.py` return the same manifest type with `modality`.

3. OCR MVP

- Implement `image_quality.py` with basic readable file, dimensions, blur placeholder, and orientation checks.
- Implement `extract_text.py` with PyTesseract text plus confidence TSV output.
- Implement `ocr_router.py` so Apple Live Text imports can be accepted as text and Tesseract can be run for image files.

4. Normalization MVP

- Implement `clean_text.py` with whitespace cleanup, answer label normalization, OCR character corrections, and section marker cleanup.
- Implement `split_lsats.py` with a conservative splitter for LR and RC.
- Implement `canonical_mapper.py` to emit the canonical schema.

5. Validation MVP

- Implement missing-field checks, five-choice checks, OCR confidence checks, and duplicate hash checks.
- Implement `review_routing.py` to write bad records into reason-specific folders.

6. Classification placeholder

- Implement `chunking.py` with tokenizer-based token counts and chunks.
- Implement `classify_document.py` as a stub or simple baseline that records strategy and placeholder labels.
- Add tests proving long inputs choose chunked strategy.

7. API wrapper

- Wire `api/routes.py` to call `run_image_lane` and `run_text_lane`.
- Keep the API thin. Do not put parsing logic inside route handlers.

Three-session next-build sequence

Session 1: Canonical records and deterministic text lane

- Build the canonical schema, Pydantic models, `text_loader.py`, `clean_text.py`, `split_lsats.py`, and `canonical_mapper.py`.
- Add fixtures for generated LR and RC text.
- Add tests for answer choice extraction, missing fields, and malformed splits.
- Deliverable: `data/raw_text/sample.txt` can produce one or more valid JSON records in `data/normalized/records/`.

Session 2: Image lane and OCR quality loop

- Build `image_loader.py`, `image_quality.py`, `extract_text.py`, and `ocr_router.py`.
- Store OCR text, confidence TSV, and OCR manifest under `data/ocr_output/`.
- Route low-confidence or malformed OCR records to `data/review_queue/`.
- Deliverable: one iPhone screenshot can flow into the same canonical schema as generated text.

Session 3: Classification and retrieval-ready persistence

- Add token-length measurement across `data/normalized/`.
- Implement straight, head-tail, and chunked strategy selection in `chunking.py` and `classify_document.py`.
- Add filesystem persistence first, then PostgreSQL adapter if needed.
- Add FastAPI endpoints for ingesting, retrieving records, and listing review queue items.
- Deliverable: normalized LSAT records are classified, validated, deduped, persisted, and queryable through the API.